如何使用 MySQL 的 EXPLAIN 语句进行查询分析？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，18647+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

回答重点

EXPLAIN 是 MySQL 分析 SQL 执行计划的核心工具，在 SELECT 语句前加上 EXPLAIN 就能看到优化器打算怎么执行这条查询。重点关注这几个字段：

1）**type** 访问类型，直接决定了查询效率。性能从高到低排列：const > eq_ref > ref > range > index > ALL。看到 ALL 就意味着全表扫描，数据量一大性能就崩

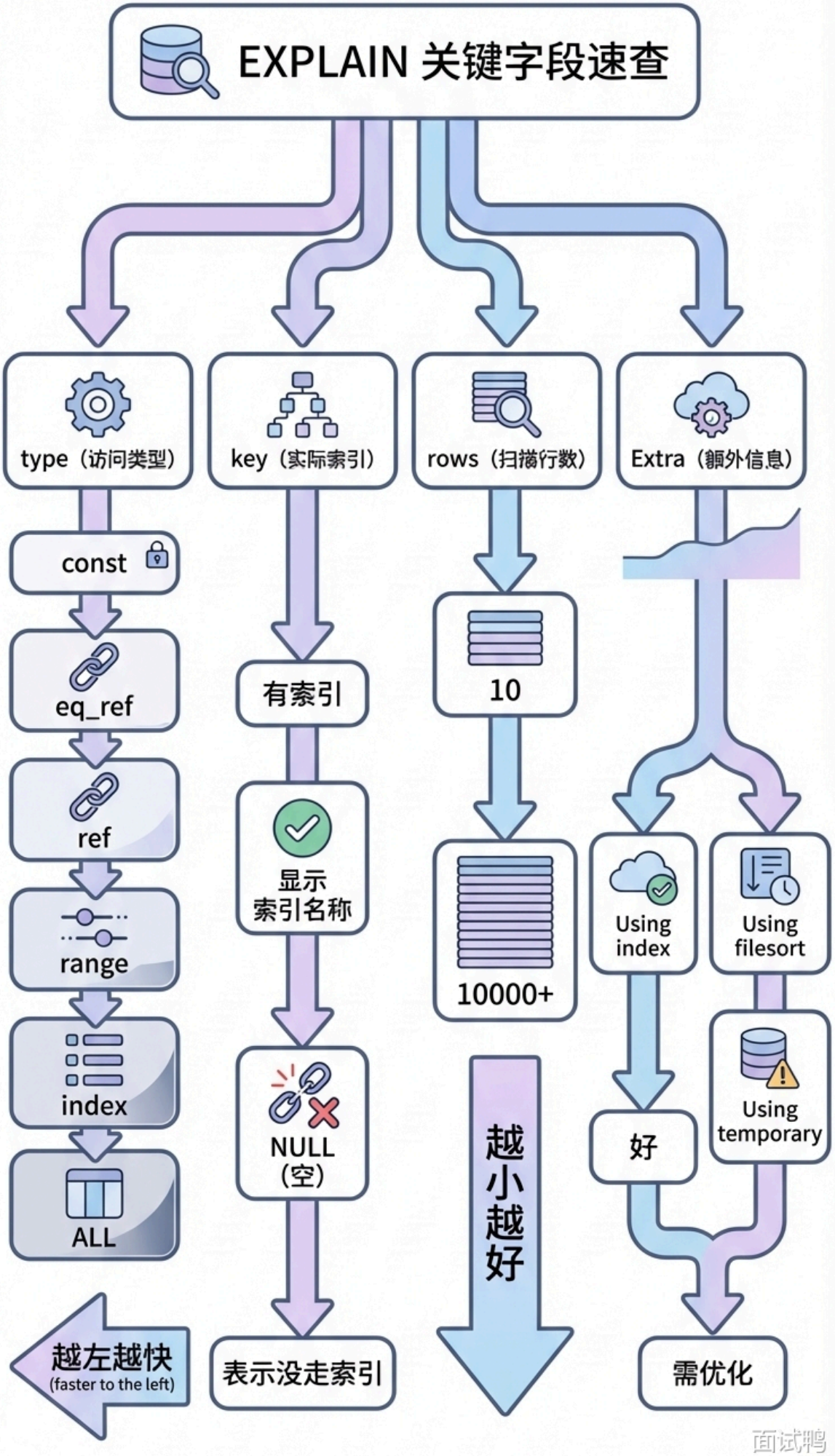
2）**key** 实际用到的索引。possible_keys 只是候选，key 才是优化器最终选的。如果显示 NULL 就说明压根没走索引

3）**rows** 预估要扫描的行数。这个值越小越好，几百行和几十万行的差距可能是毫秒和几秒的差别

4）**Extra** 额外信息藏着很多细节。Using index 说明走了覆盖索引，数据直接从索引拿不用回表；Using filesort 说明排序没法用索引得额外排序；Using temporary 说明用了临时表，这两个一般都需要优化

type 的每个级别具体含义：

- const：主键或唯一索引等值查询，最多一行匹配，最快
- eq_ref：连接查询里通过主键或唯一索引关联，每次只取一行
- ref：用非唯一索引查，可能返回多行
- range：索引范围扫描，用在 BETWEEN、>、< 这类条件上
- index：扫描整棵索引树，比 ALL 好但还是很慢
- ALL：全表扫描，遇到大表直接起飞


EXPLAIN 关键字段速查
type（访问类型）
key（实际索引）
rows（扫描行数）
Extra（额外信息）
const
eq_ref
ref
range
index
ALL
有索引
显示
索引名称
NULL
（空）
10
10000+
越小越好
Using
index
Using
filesort
Using
temporary
好
越左越快
(faster to the left)
表示没走索引
需优化

面试鸭

mansmya.com

扩展知识

实战案例：从 5000 行扫描优化到 500 行

假设有张 employees 表：

```
▼sql复制代码CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department_id INT,
    salary DECIMAL(10, 2),
    hire_date DATE,
    INDEX (department_id)
);
```

要查部门 ID 为 5 且薪水在 50000 到 100000 之间的员工，按薪水降序：

```
▼sql复制代码EXPLAIN SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE department_id = 5 AND salary BETWEEN 50000 AND 100000
ORDER BY salary DESC;
```

执行计划输出：

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employees	ref	department_id	5000	Using where; Using filesort

问题一眼就能看出来：走了 department_id 索引但 rows 是 5000，而且 Extra 里有 Using filesort，说明虽然过滤用了索引，排序却要额外做一次文件排序。

优化方案是建一个复合索引，把 department_id 和 salary 都包进去：

```
▼sql复制代码CREATE INDEX idx_department_salary ON employees (department_id,
salary);
```

再跑一次 EXPLAIN：

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employees	range	idx_department_salary	500	Using where

rows 从 5000 降到 500，Using filesort 也没了。因为复合索引里 salary 已经排好序了，查询直接顺着索引拿数据就行，不用再单独排一遍。

select_type 常见类型

- SIMPLE：简单查询，没子查询没 UNION
- PRIMARY：最外层的查询
- SUBQUERY：SELECT 或 WHERE 里的子查询
- DERIVED：FROM 子句里的子查询，MySQL 会把它物化成临时表
- UNION：UNION 里第二个及之后的 SELECT
- UNION RESULT：UNION 的结果集

EXPLAIN 的两个增强版本

MySQL 5.6+ 支持 `EXPLAIN FORMAT=JSON`，输出更详细的 JSON 格式，能看到每一步的成本估算。MySQL 8.0.18+ 还新增了 `EXPLAIN ANALYZE`，这个更厉害，会真正执行查询并给出实际的执行时间和行数，不只是预估值。调试复杂查询的时候特别有用。

```
▼sql复制代码-- JSON 格式，能看到 cost 成本
EXPLAIN FORMAT=JSON SELECT * FROM employees WHERE department_id = 5;

-- 真正执行并返回实际耗时
EXPLAIN ANALYZE SELECT * FROM employees WHERE department_id = 5;
```

面试官追问

提问：一个慢查询，EXPLAIN 看到走了索引但还是慢，可能是什么原因？

> 详细解答可以看：[MySQL 中使用索引一定有效吗？如何排查索引效果？]

回答：几种可能。第一，回表次数太多，虽然走了二级索引，但查出来几十万行每行都要回表，随机 IO 吃不消，还是慢。第二，索引选择性差，比如 status 字段只有几个值，走索引过滤效果不好。第三，数据量太大，索引树本身就很大，内存装不下频繁读磁盘。第四，EXPLAIN 是估算，实际执行可能和估算不一样，可以用 EXPLAIN ANALYZE 看真实执行情况。第五，返回数据量大，网络传输耗时。重点看 EXPLAIN 的 rows 估算值和 Extra 列有没有 filesort、temporary。

提问：possible_keys 有值但 key 是 NULL，这是什么情况？

回答：说明优化器认为全表扫描比走索引更划算。常见场景是表数据量很小，或者查询条件覆盖了大部分数据，走索引反而多一次回表的开销。另外索引字段类型不匹配也会导致索引失效，比如 varchar 字段拿 int 去比较。

提问：Using index 和 Using index condition 有什么区别？

回答：Using index 是覆盖索引，查询需要的字段全在索引里，直接从索引拿数据不用回表。Using index condition 是索引条件下推 ICP，MySQL 5.6 引入的优化，把部分 WHERE 条件下推到存储引擎层在索引遍历时就过滤掉，减少回表次数。前者是完全不回表，后者是少回表。

提问：怎么判断一个索引的选择性好不好？

回答：用 `SELECT COUNT(DISTINCT column) / COUNT(*)` 算一下这个字段的区分度。比如订单表的 order_id 区分度接近 1，状态字段可能只有 0.001，那给状态字段单独建索引基本没意义。一般区分度低于 0.1 的字段不适合单独建索引，但可以作为复合索引的一部分。